The entire development process of RAG

GitHub Link: https://github.com/wdwe123/climate-policy-rag

Step 0: Environment & Requirements

1. The OCR and Chunking processes are running on the main computing nodes of NYU Greene HPC as follows:

Component	Version / Description
Python	3.10
PaddlePaddle	2.6 (GPU version, CUDA 11.8)
PaddleOCR	2.7 (PP-OCRv3 English recognition model)
pypdfium2	4.x (PDFium backend, high-DPI rendering)
OpenCV	4.8 (cv2, used for table line detection)
tiktoken	0.6 (BPE tokenization, cl100k_base encoding)
numpy / pandas	1.26 / 2.0
portkey-ai	1.x (LLM API gateway, routed to Vertex AI)
GPU	NVIDIA V100 32GB

2. The retrieval and Streamlit application are run in a local Windows environment. The dependencies are listed in `requirements.txt`.

Step 1: Getting Data & Prepare

1. Using Google's advanced search function to find documents
   - **Example**:
     site:az.gov OR site:azleg.gov
     (tribal OR tribe)
     (drought plan OR climate plan OR water strategy)
     filetype:pdf

2. Record documents' URL, Policy Level, File Name, Geographic Information (State name; County Name; etc.). See Figure 1.

	State	County	City	Tribe Name	Policy Level (Federal/...	Policy Name	Policy Type (Climate/Energy/Weather/et	Links to PDF
2	New Mex...				State	NEW MEXICO CLIMATE ADAPTATION AND	Climate	https://www.env.nm.gov/climateaction/wp-content/uploads/sites/39/2024/07/NM-C.A.R.P._03
3	New Mex...	Taos		Taos Pueblo	Tribe		Collaborate with various types of enviror	https://taosunited.org/what-we-do-2/environment-energy/
4	New Mex...	Taos		Picuris Pueblo	Tribe	Solar Energy Implementation Strategies on PI	Energy	https://www.energy.gov/sites/default/files/2022-12/picuris_pueblo_hammond_sand2022-166
5	New Mex...	Rio Arriba		Ohkay Owingeh	Tribe	Legar Fernández, Heinrich Introduce Legislati	Water	https://fernandez.house.gov/news/documentsingle.aspx?DocumentID=303
6	New Mex...	Rio Arriba		Santa Clara Pueblo	Tribe	Climate Change Health Impacts and Commur	Climate	https://www.nmlegis.gov/handouts/WNR%20072224%20Item%205%20Alliance%20of%20H

Figure 1

3. *Optional*: *Pre-screening for Complex PDF Layouts*
   - Quick scan each PDFs, flag those with complex layouts (e.g., multi-column text, dense tables, or heavy images) that may hinder OCR performance.

- Move flagged files into a separate table (e.g., `unstructured_files.csv`).
- Handle them later with **manual chunking**.

4. Download all PDF and the metadata to local. Record the local PDF path in metadata. Format can be referenced as Figure 2.

5. PDF Validation:
   - **File Size**: < 1KB will be marked as “invalid file”
   - **Magic number check**: Verify the PDF header (%PDF-)
   - **Check page number**: if page number = 0, marks as “invalid file”
   - **File readability**: Attempt to open the file using pypdfium2

6. Deduplication:
   - **Path deduplication**: The file path is exactly the same, only keep one file
   - **Content-based deduplication:** Using SHA256 hash

7. Geographic Metadata Merging:
   - The same document may appear multiple times in the metadata (from different state/county/tribal policy databases), and the geographical labels included each time are different.
   - After deduplication, the geographical information in multiple lines needs to be merged into one line to avoid losing the coverage range.
   - **Example**:
     - A federal document appears in both Arizona and Oklahoma datasets.
     - After merging: `state_list = ["Arizona", "Oklahoma"]`
   - **Output**:
     - Intermediate file: `policy_metadata_2_merged.csv`
     - After manual verification, the final version: `policy_metadata_4.csv`

   * If you did “*3. Optional: Pre-screening for Complex PDF Layouts*”, do the same thing to `unstructured_files.csv`, output `unstructured_metadata.csv`

	A	B	C	D	E	F	G	H	I	J	K
1	row_index	policy_id	source_url	file_path	state_list	county_list	city_list	tribe_list	policy_leve	policy_type	
2	0	f1c94ead9	https://ww	E:/2026_ca	['Arizona']	[]	[]	[]	State	Drought Plan	
3	1	dd08b0c9(	https://der	E:/2026_ca	['Arizona']	[]	[]	[]	State	Hazard Mitigation Plan	
4	2	12d280431	https://resi	E:/2026_ca	['Arizona']	[]	[]	[]	State	Climate Action Plan	
5	3	7e74dc1b1	https://der	E:/2026_ca	['Arizona']	[]	[]	[]	State	Emergency Plan	
6	4	62a437cab	https://wat	E:/2026_ca	['Arizona']	[]	[]	[]	State	Water banking	
7	5	72c61141b	https://ww	E:/2026_ca	['Arizona']	['Apache']	[]	[]	County	Hazard Mitigation Plan	
8	6	e08f04851	https://ww	E:/2026_ca	['Arizona']	['Cochise']	[]	[]	County	Hazard Mitigation Plan	
9	7	467df4634	https://ww	E:/2026_ca	['Arizona']	['Coconino	[]	[]	County	Hazard Mitigation Plan	
10	8	df475d575	https://cm	E:/2026_ca	['Arizona']	['Gila']	[]	[]	County	Hazard Mitigation Plan	
11	9	50c170e94	https://cor	E:/2026_ca	['Arizona']	['Pima']	[]	[]	County	Hazard Mitigation Plan	
12	10	72734cf39	https://ww	E:/2026_ca	['Arizona']	['Pinal']	[]	[]	County	Hazard Mitigation Plan	
13	11	fd9e6ccff1	https://ww	E:/2026_ca	['Arizona']	['Santa Cru	[]	[]	County	Hazard Mitigation Plan	
14	12	c236dbecc	https://ww	E:/2026_ca	['Arizona']	['Yavapai']	[]	[]	County	Hazard Mitigation Plan	
15	13	3246a4cf0	https://ww	E:/2026_ca	['Arizona']	['Yuma']	[]	[]	County	Hazard Mitigation Plan	
16	14	888777258	https://ww	E:/2026_ca	['Arizona']	['La Paz']	[]	[]	County	Hazard Mitigation Plan	
17	15	b4339706b	https://ww	E:/2026_ca	['Arizona']	['Maricopa	[]	[]	County	Hazard Mitigation Plan	
18	16	66ccce1bf	https://ww	E:/2026_ca	['Arizona']	['Mohave']	[]	[]	County	Hazard Mitigation Plan	
19	17	946264d9e	https://ww	E:/2026_ca	['Arizona']	['Navajo']	[]	[]	County	Hazard Mitigation Plan	
20	18	1811f1cb5	https://ww	E:/2026_ca	['Arizona']	['Cochise']	[]	[]	County	Comprehensive Plan	
21	19	3f7b28443	https://ww	E:/2026_ca	['Arizona']	['Cochise']	[]	[]	County	Emergency Plan	

Figure 2: policy_metadata_4.csv

Step 2: OCR & Chunking

1. Load Metadata:

   Each row of metadata contains:

Field	Type	Description
policy_id	str	SHA1 hash (unique file identifier)
source_url	str	Original download URL
file_path	str	Local file path
policy_level	str	Tribal / County / City / State / Federal
policy_type	str	Hazard Mitigation Plan / Climate Action Plan / ...
state_list	list	["Arizona"], etc.
county_list	list	
city_list	list	
tribe_list	list	["Navajo Nation", ...]

2. Render PDFs into images:
   1. Tool: pypdfium2
   2. Parameters:
      - DPI = 200
      - scale = DPI / 72

3. PaddleOCR Text Recognition:
   1. Model: PaddleOCR PP-OCRv3
   2. Parameters:

- use_angle_cls = False
- lang = "en"
- drop_score = 0.80
- use_gpu = True

3. OCR Quality Statistics (per page):
   - ocr_conf_mean: The average confidence level of all recognition results on the page
   - ocr_low_conf_ratio: The proportion of results with a confidence level less than 0.85

4. Multi-Column Aware Line Merging:
   1. **Function:** merge_ocr_items_into_lines_with_pos()
   2. **Core Problem:**
      PaddleOCR returns text as individual bounding boxes grouped by coordinates.
      In multi-column layouts, text from the left and right columns may fall within the same y-band, causing them to be incorrectly merged into a single line and resulting in disordered text.
   3. **Solution:**
      - **Group into Lines by Y-axis (y_threshold = 12 px)**
        - Sort all OCR boxes by y_min
        - If the difference in y_min between boxes is ≤ 12 pixels, assign them to the same line
        - Each line stores: {y_min, y_max, x_min, x_max, items}
      - **Detect Column Gaps and Split Columns (col_gap_min_px = 60 px)**
        - Within each line, sort OCR boxes by x_min
        - Compute gap: gap = current_box.x_min - previous_box.x_max
        - If gap ≥ 60 px, treat it as a column break and split the line into separate column segments
        - Each segment is treated as an independent “line”
      - **Global Sorting**
        - Sort all lines (including split segments) by (y_min, x_min)
        - For text at the same vertical level, left columns are prioritized over right columns
        - This preserves a natural **top-to-bottom, left-to-right reading order**

5. Header Detection & Blacklist:
   1. **Objective:** Automatically identify document titles or running headers that repeatedly appear at the top of pages (e.g., *"Arizona Climate Action Plan"*) and remove them to prevent interference with chunk content.
   2. **Methodology:**
      - For each page, extract all text lines within the top 12% of the page (HEADER_BAND_FRAC = 0.12)
      - Count the frequency of each text line across all pages
      - Starting from page 2, build a blacklist:
        - Lines that appear in ≥ 40% of pages (HEADER_FREQ_FRAC = 0.40)

- And have length ≥ 6 characters

3. **Usage:**
   - During subsequent heading detection and region segmentation, blacklisted lines are skipped
   - They are not considered as candidate headings and are excluded from chunk content

6. Page Classification

6.1: TOC (Table of Contents) Detection

- **Function:** is_toc_like_page_strong()
- Compute a **TOC score** using five weak signals:

Signal	Weight	Description
Keyword match	1.2	Contains "TABLE OF CONTENTS", "CONTENTS", etc.
Dot leader ratio	1.0	Presence of patterns like 23
Line-ending number ratio	0.9	Lines ending with page numbers
TOC entry ratio	0.9	Lines with SECTION/CHAPTER/number prefix + trailing number
Short line ratio	0.6	Proportion of lines with 8–70 characters

- **Decision Rules:**
  - If kw = 1 AND (endnum_ratio ≥ 0.12 OR entry_ratio ≥ 0.10 OR dots_ratio ≥ 0.05) → classify as **TOC**
  - OR if toc_score ≥ 1.65 → classify as **TOC**
  - Only the **first TOC page** is retained; subsequent TOC pages are skipped

- **Handling:**
  - TOC pages are labeled as is_toc = True
  - Skipped and Excluded from chunking

6.2: **Figure Page Detection**

- **Function:** is_figure_like_strong()
- **Methodology:**
  - OCR text coverage ≤ 5% of the page area
  - ink_ratio (non-white pixel ratio) ≥ 18%
  - Short-token ratio ≥ 75% (many map labels, very short words)
  - Median OCR token length ≤ 10 characters
- **Decision Rule:**
  - If combined score ≥ 1.6 → is_figure = True
  - Skipped and excluded from chunking

6.3: **Form / Questionnaire Page Detection**

- **Function:** is_form_like_page_strong()
- **Methodology:**
  - **Keywords:** FORM / QUESTIONNAIRE / SURVEY / CHECKLIST / APPLICATION
  - **Field-like patterns:** structured formats (e.g., labeled fields, placeholders)

- **Underline / blank patterns:** sequences like ___, □, or repeated blanks
- **Checkbox / selection patterns:** [ ], □, YES / NO

- **Decision Rule:**
  - If `form_score ≥ 1.55` OR (`kw = 1` AND any signal exceeds threshold) → `is_form = True`
  - Skipped and excluded from chunking

6.4: **Table Page Detection**

- **Tools & Methodology:**
  - Grid-based detection (`detect_table_grid_lines()`, OpenCV)
    - Detect horizontal/vertical lines via binarization + morphology
    - Compute grid density, intersections, and bounding box area
    - If thresholds are met → classify as table page
    - Suitable for bordered tables (e.g., Word-exported)
  - OCR-based detection (`detect_table_block_from_ocr()`)
    - Group OCR boxes by rows and detect column alignment
    - ≥ 6 consistent rows → classify as table region
    - Suitable for borderless tables

6.5: **Cover Page Detection**

- A page is classified as a cover if **≥ 2 conditions** are met:
  - Total page count ≤ 15
  - Located within the first 5 pages
  - Contains document title keywords
  - OCR text coverage ≤ 20% (large-font dominant)
- Labeled as `is_cover = True`
- Skipped and excluded from chunking

7. Table Extraction & Reconstruction

7.1: **OpenCV-based Grid → Cell Extraction**

- **Function:** `detect_table_cells_from_image()`
- Returns `found = True` when:
  - Detect horizontal/vertical line projections within the table bounding box
  - Use peaks to define row/column boundaries and generate cell boxes (x0, y0, x1, y1)
  - Merge OCR text within each cell (sorted by y → x)
  - Output as a Markdown table

7.2: **OCR-based x/y Clustering**

- **Function:** `detect_table_block_from_ocr()`
- For borderless tables:
  - Cluster OCR boxes by x_center (tol ≈ 35 px) to identify columns
  - Cluster by y_center to identify rows
  - Build a matrix [row, col] → text, separated by delimiters

- Merge sparse rows if needed

7.3: **LLM Fallback (GPT-4o-mini via Portkey)**

- Triggered only when both methods fail
- LLM takes OCR text as input and outputs structured Markdown tables.

8. Heading Detection

8.1 **Three Heading Types**

- **Function**: `title_like()`
  - 4–120 characters
  - 2–12 words
  - Mostly capitalized words
  - No URLs or excessive punctuation
  - Numeric ratio < 18%
- **Numeric Headings**
  - Examples: `2.1 Water Conservation Goals` ; `4.2.3 Implementation Timeline`
  - Rules:
    - Appeared near the top 30% of the page AND left-aligned
    - Must pass `title_like()` validation to avoid false positives such as “1.7 million customers”
- **Word-based Headings**
  - Examples: `SECTION 3: Water Resources` ; `CHAPTER IV Environmental Planning`
  - Rules:
    - Must begin with keywords such as CHAPTER, SECTION, or PART
    - Usually followed by Roman numerals or numeric identifiers
    - Usually left-aligned AND surrounded by large vertical gaps
- **Title-style Headings**
  - Examples:
  - Rules:
    - Located within the top 35% of the page, OR surrounded by large vertical gaps
    - Must pass `title_like()` validation
    - Single-word headings (e.g., “Introduction”, “Overview”) are detected only if included in a predefined whitelist

8.2 **False Positive Filtering**

- Skip candidate headings if they:
  - Appear in footer regions
  - Are likely table rows (currency, dates, heavy numeric content)
  - Match running headers in `header_blacklist`
  - Contain URLs or excessive punctuation
  - Exceed 140 characters
  - Overlap with detected table regions

8.3 **Deduplication & Limiting**

- Remove duplicate headings with similar text and nearby positions (±6 px)
- Prioritize numeric and word-based headings over title-style headings
- Limit the number of headings per page to avoid over-segmentation

8.4 **LLM-assisted Heading Processing**

Triggered under three conditions:

- **Heading Normalization**
  - Function: `llm_normalize_heading()`
  - Condition: The heading has abnormal formatting (e.g., `"None: Water Goals"`)
  - Prompt:

```
You normalize document section headings.
Return ONLY JSON with keys:
- canonical: string (clean, human-readable)
- level: integer 1-3 (1=top, 2=subsection, 3=subsubsection)
- drop: boolean (true only if this is a running header/footer or obvious noise)
Rules:
- Remove leading 'None:' or similar artifacts.
- Keep meaningful numbering (e.g., '2.1', 'SECTION 3') when present.
```

- **Section Boundary Repair**
  - Function: `llm_boundary_repair()`
  - Condition: No heading is detected, but page content strongly suggests the start of a new section.
  - Prompt:

```
You detect whether a new major section starts on this page and propose the best section heading.
Return ONLY JSON with keys:
- new_section: boolean
- canonical: string (only if new_section=true)
- level: integer 1-3 (only if new_section=true)
Guidelines:
- Prefer true only when there is strong evidence of a new section heading.
- If the page continues the previous section, return
```

- **Dense Heading Grouping**
  - Function: `llm_group_dense_headings()`
  - Condition: A page contains too many tiny heading regions (≥ 7 heading regions; OR tiny-region ratio ≥ 55%)
  - Prompt:

```
You help reduce over-fragmentation caused by many tiny subheadings
on one page.
You will receive a list of regions in order. Each region has:
- id (int)
- heading (string)
- tokens (int)
- preview (string) (first 1-2 lines)
Task:
Group CONSECUTIVE regions into a smaller number of groups suitable
for RAG chunks.
Return ONLY JSON:
{"groups": [{"region_ids": [0,1,2], "section_name": "...", "level":
1|2|3}, ...]}
Rules:
- Every region id must appear exactly once, in order.
- Prefer 2-4 groups if there are many regions.
- If headings are minor (tiny content), it is OK to group under a
parent-like section_name.
- Do NOT invent content beyond headings; section_name should be
```

9. Split Page into Heading Regions

   Function: split_page_into_heading_regions()

   Using the headings detected in Step 8, divide OCR text on each page into semantic regions.

   Algorithm:

   - Sort heading boxes by y_min to determine vertical ranges
   - Assign OCR lines between adjacent headings into the corresponding region
   - Remove duplicated heading text from region content
   - Skip lines matching the running-header blacklist
   - Replace table-region lines with reconstructed table_block_text to avoid duplication

   Fallback:

   - If no heading is detected on the page, treat the entire page as a single region
   - Continue using the current active section (current_section) from previous pages

10. Appendix Detection

    Detect appendix sections using keywords within the first ~12 OCR lines.

    - **Function**: detect_appendix_on_page()
    - **Keywords**: APPENDIX, ANNEX, EXHIBIT, ATTACHMENT
    - **False Positive Prevention**: If ≥ 2 appendix-like entries appear on the same page, treat it as an appendix TOC page instead of a real appendix section
    - Handling:
      - is_appendix = True

- appendix_label = "APPENDIX A"
- Retrieval priority reduced

11. Cross-page Chunk Accumulation

    Maintain a rolling chunk buffer while traversing pages.

    - **Parameters:**
      - TARGET_TOKENS = 1200
      - OVERLAP_TOKENS = 120
    - **Rules:**
      - Add region text into the current chunk buffer
      - If token count exceeds TARGET_TOKENS:
        - Output current chunk
        - Keep the last OVERLAP_TOKENS as overlap for the next chunk
      - Force chunk split when major section changes occur
      - Special pages (TOC / Figure / Form / Table) are emitted independently
    - **PageChunk Structure (!! IMPORTANT !!):**

```
@dataclass
class PageChunk:
doc_title: str
section: str
section_path: List[str]
pages: str
text: str
tokens: int

pdf_path: str
page_type: str

is_toc: bool
is_figure: bool
is_form: bool
is_table: bool
is_cover: bool
is_appendix: bool

appendix_label: Optional[str]

# metadata
policy_id: str
source_url: str
policy_level: str
policy_type: str

state_list: List[str]
county_list: List[str]
city_list: List[str]
tribe_list: List[str]
```

12. Small Chunk Merging

Merge overly small chunks generated after region splitting

- **Function:** merge_small_chunks()
- **Parameters:**
  - MIN_CHUNK_TOKENS = 250
  - MERGE_TARGET_TOKENS = 650
  - MERGE_MAX_TOKENS = 1100
- Merge Conditions:

  Merge only if all conditions are satisfied:
  - Same document (pdf_path and doc_title)

- Same appendix status
- Similar or parent-child section_path relationship
- Merged token count ≤ MERGE_MAX_TOKENS
- The chunk type is NOT: TOC / Figure / Form / Table / Cover

13. Metadata Attachment & JSONL Output

Each PageChunk already contains document-level metadata attached in Step 11, such as policy_id, source_url, and policy_level.

- **Output format:** One JSON object per line (JSONL)
- Output file: chunking_output/script_chunking_v11_6.jsonl
- Diagnostics file: chunking_output/chunking_diagnostics_v11_6.csv
  - Each row represents one PDF and records:
    - Total pages and processed pages
    - Number of TOC / Figure / Form / Table / Cover pages
    - Total chunks and average token count
    - Number of LLM calls by type
    - Average OCR confidence

14. Optional: Manual Chunking

Used for PDFs with highly complex layouts, poor OCR quality, vertical text, handwriting, or special formatting.

- **Script:** manual_chunker.py
- **Interface:** Streamlit
- **Workflow**:
  - Select a target document from unstructured_metadata.csv (See **Step 1: Getting Data & Prepare, Section 7. Geographic Metadata Merging**)
  - Open the PDF manually using the default PDF viewer
  - Enter section name, text content, page number, page type, and appendix status in Streamlit
  - Click **"Add Chunk"**; the system calculates token count and generates JSONL records, e.g. manual_chunking.jsonl
  - Merge manual_chunking.jsonl and script_chunking_v11_6.jsonl name as "all_chunks_v11_6.jsonl"

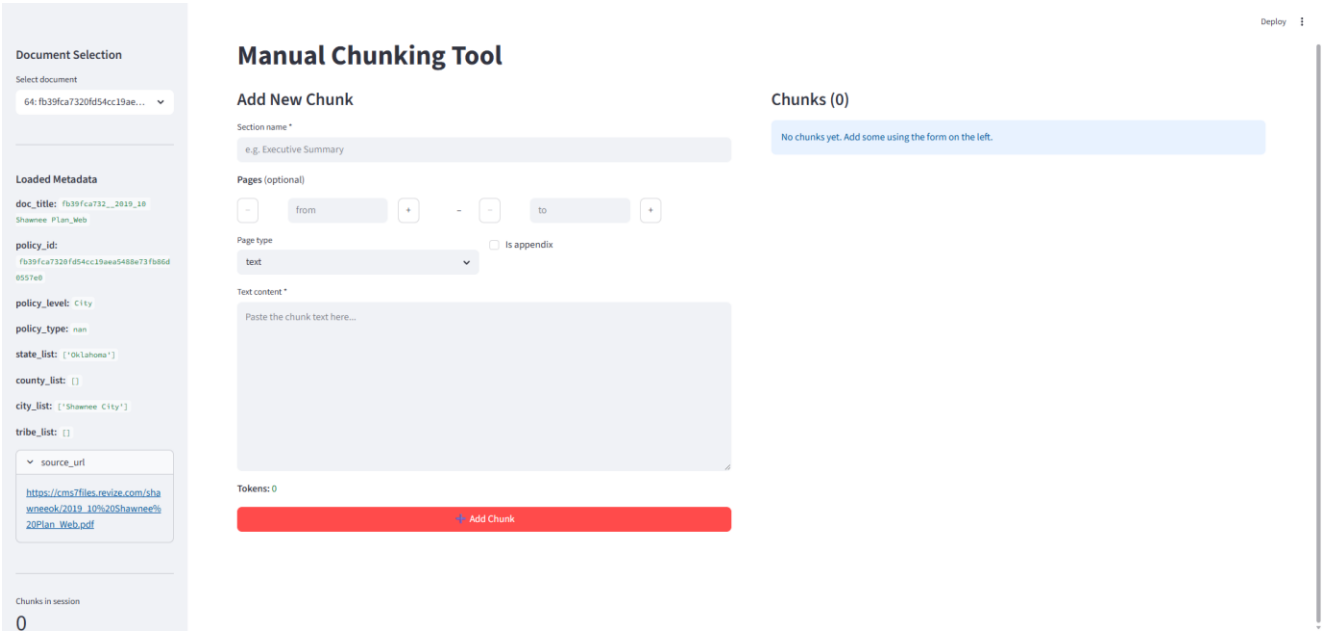

Figure 3: Manual Chunking Interface

15. LLM Semantic Classification

Script: classify_chunks.py

Model: claude-haiku-4-5 via NYU Portkey

15.1 **Rule-based Fast Classification**

Condition	**Direct Classification**
is_toc = True	administrative
is_cover = True	administrative
len(text) < 30	noise
is_table = True AND len(text) < 80	noise

15.2 **LLM Classification**

Prompt:

```
You are classifying chunks from US climate policy documents for a RAG system
serving Native American communities in Arizona, New Mexico, and Oklahoma.

For each chunk, output ONLY a JSON object with these fields:
{
  "primary_tag": "<one of the 8 tags below>",
  "secondary_tags": ["<optional>", "<optional>"],
  "policy_score": <float 0.0-1.0>
}

PRIMARY TAG OPTIONS (choose exactly one):
- action_policy: Direct actionable policy text — regulations, requirements,
goals, rules, mandates
- implementation: Implementation details — timelines, responsible parties,
procedures, steps
- funding_program: Funding, grants, financial assistance programs,
application processes, budgets
- background_context: Background, risk assessment, situation overview,
statistics, history (not directly actionable)
- table_data: Structured tables with meaningful data (risk matrices,
statistics, funding allocations)
- reference_appendix: Definitions, acronym lists, bibliographies, appendix
indexes, legal citations
- administrative: Cover pages, signatures, acknowledgments, meeting minutes,
contact info, TOC
- noise: Garbled OCR, near-empty content, repeated headers/footers,
meaningless fragments

SECONDARY TAGS: 0-2 additional tags that also apply (use [] if none).

POLICY_SCORE: How useful is this chunk for answering policy questions?
  1.0 = directly answers "what policy applies / what can I apply for"
  0.0 = noise or purely administrative

Output ONLY the JSON. No explanation.
```

Output:

- 8 primary tags
- Policy Score (0 - 1)

15.3 **Retrieval Tier Mapping**

The `retrieval_tier` is automatically assigned based on `policy_score`:

policy_score	retrieval_tier	Description
≥ 0.75	primary	Used in retrieval, weight ×1.3
≥ 0.40	secondary	Used in retrieval, weight ×1.0
≥ 0.10	low_priority	Used in retrieval, weight ×0.6
< 0.10	exclude	Excluded from retrieval

15.4 Output Schema
Each chunk receives four new fields:

```
{
  "primary_tag": "action_policy",
  "secondary_tags":
["implementation"],
  "policy_score": 0.92,
  "retrieval_tier": "primary"
```

Output file: chunking_output/all_chunks_classified.jsonl

16. Filter out chunks that have been labeled as "exclude"

17. Final Chunk Statistics

Metric	Value
Processed PDFs	196
Total chunks	~10,030
Average chunk size	~700 tokens
Primary-tier chunks	~45%
Secondary-tier chunks	~30%
Exclude / noise chunks	~8%

Step 3: Embedding & Indexing

1. Deterministic Chunk UUID Generation
   Generate a unique UUID string for each chunk, used as the **Qdrant point ID**.
   The UUID is created by hashing the combination of policy_id, pages, and row index _idx using MD5.

2. Build Embedding Input Text
   Each chunk's embedding text is constructed by concatenating three components:

- doc_title
- section
- text

Example:

```
Arizona Water Resources Management Act | Section 3: Tribal Water Rights
The following provisions apply to Indigenous communities within the State of
```

* build_bm25.py uses the same format to construct BM25 corpus text

3. Embedding with Gemini API

   Parameters:

   - Dimention: **3072**
   - Batch size: **6** (Each batch contains ~10 chunks)

4. Store Vectors into Qdrant

   Vectors are never written to disk as intermediate files — each batch is upserted into Qdrant immediately after the API returns:

   - **Stored content:** 3072-dim vector + full payload (raw text, policy_level, retrieval_tier, policy_score, etc.)
   - **Payload indexes:** 7 fields indexed (KEYWORD / BOOL / FLOAT) to enable fast pre-filtering during retrieval
   - **Checkpoint / resume:** existing Qdrant IDs are enumerated before each run; already-indexed chunks are skipped to avoid duplicate API charges

5. Build BM25 Sparse Index

   A sparse index is built in parallel with the vector index to support hybrid retrieval:

   - Uses the **exact same text format** as the Gemini embedding step (doc_title + section + text)
   - Tokenizer: re.findall(r"[a-z0-9]+", text.lower()) — lowercase alphanumeric tokens only
   - BM25Okapi(corpus_tokens), k1=1.5, b=0.75 (library defaults)
   - **ID alignment:** BM25 corpus position maps 1-to-1 to Qdrant point UUIDs via the shared make_chunk_id() function, so RRF fusion can join results from both indexes using the same ID space
   - Save as bm25_index.pkl + bm25_corpus_ids.pkl

6. Final Statistics

Metric	**Value**
Total chunks after classification	~10,030
Searchable chunks after exclude filtering	~8,500
Qdrant vector dimension	3072
BM25 corpus size	~8,500

Metric	Value
Vector storage path	qdrant_storage/
BM25 storage files	bm25_index.pkl + bm25_corpus_ids.pkl

7. Interface with Retrieval Stage
   After Qdrant and BM25 indexes are built, `retriever.py` handles retrieval:
   - **Vector search:** Qdrant cosine similarity search
   - **BM25 search:** score query tokens against `bm25` and map results via `corpus_ids`
   - **RRF fusion:** combine dense and sparse results, weighted by `retrieval_tier`
   - **LLM reranking:** score candidate chunks from 0–10 and normalize as `cross_score`

Step 4: Retrieval & Generation

1. Embed Query
   The query is embedded using the same model as the document index:
   - Model: @vertexai/gemini-embedding-001, output dim = 3072

2. Extract Geo Entities
   Return: `{tribes: [], counties: [], cities: [], states: []}`.
   Methods:
   - Rule-based (primary): vocabulary built from `policy_metadata_4.csv` at startup. Longest-match first to prevent partial overlaps (e.g. "Apache" matching inside "White Mountain Apache"). Alias normalization applied before matching (e.g. "Navajo" → "Navajo Nation").
   - LLM fallback: if rule-based returns all-empty, Claude Haiku is called with a structured prompt: `"Return ONLY a JSON object with keys: tribes, counties, cities, states."` Response parsed with `re.search(r'\{.*\}')`.

3. Determine Active Geographic Levels
   Start level is set by the most specific entity found:

Geo entities present	Start level	Levels searched
tribes	tribe	tribe → county → state → federal
cities	city	city → county → state → federal
counties	county	county → state → federal
states	state	state → federal
none	federal	federal only

4. Vector Search (per level)
   For each active level, a Qdrant `query_points` call is made with a compound filter:

```
Filter(must=[
    FieldCondition("policy_level",
MatchAny(["Tribe"])),
    FieldCondition("tribe_list",      MatchAny(["Navajo
Nation"])),
], must_not=[
```

- limit = top_k_per_level (default 8)
- Distance metric: Cosine
- All levels run in parallel via ThreadPoolExecutor(max_workers=len(active_levels))

5. BM25 Sparse Search (per level)
   Query tokenized identically to index construction:

```
tokens      =      re.findall(r"[a-z0-9]+",
query.lower())
```

   - Only chunks with score > 0 are returned
   - **ID whitelist**: BM25 results filtered to IDs already seen in the vector search results, ensuring both paths operate on the same candidate pool

6. RRF Fusion + Tier Weighting
   Formula:
   - RRF_score = Σ 1 / (60 + rank_i + 1)
   - final_score = RRF_score × TIER_WEIGHT
   - TIER_WEIGHT = {"primary": 1.3, "secondary": 1.0, "low_priority": 0.6, "exclude": 0.0}

   Chunks appearing in both result lists accumulate scores from both; chunks from high-quality tiers (primary) are boosted. Results sorted by final_score descending.

7. LLM Reranking
   All level candidates (capped at 25 total) are sent to Claude Haiku in a single call:

```
Rate each passage's relevance to the query on a scale
of 0-10.
```

   Raw scores normalized: cross_score = score / 10.0

   Per-level has_results flag:
   has_results = len(chunks) > 0 and max(cross_score) >= 0.2

   Fallback on parse failure: cross_score = 0.5, original RRF order preserved.

8. Answer Generation
   Top chunks from each has_results=True level (default number is 2) are passed to Claude Sonnet 4.6.
   The system prompt instructs the model to structure the answer by geographic level and cite

every claim with [Doc N] notation. Output includes a Sources section with URLs.